



GIESSWEIN
Software-Solutions

JIT, GC & Threads

Die .NET Runtime, die du jeden Tag nutzt (aber kaum kennst)

Ing. Christian Giesswein, MSc.

www.software.tirol

@giessweinweb

Ing. Christian Giesswein, MSc.

Trainings | Consulting | Entwicklung

www.software.tirol



- Ing. Christian Giesswein, MSc.
 - www.software.tirol
 - christian@software.tirol
- .NET Allrounder
 - C# & .NET, .NET Core
 - Datenbankentwicklung - EF / EF Core
 - Anwendungsentwicklung – WPF & Blazor
- Konferenz- und Keynotesprecher im DACH Raum
- Fachautor (ct, dotnetPro, Visual Studio 1, t3n)
- Gründer „Giesswein Software-Solutions“ in Tirol / Österreich (2013)
 - .NET (Core) Beratung, Schulung und Entwicklung
 - Firmensitz in Tirol / Wildschönau
 - Produktentwicklung

Microsoft
CERTIFIED
Professional

Microsoft
CERTIFIED
Trainer

Microsoft
CERTIFIED
Solutions Developer
Windows Store Apps

Microsoft
CERTIFIED
Technology
Specialist

Microsoft
CERTIFIED
Professional
Developer

Microsoft
CERTIFIED
Solutions Developer
Web Applications

.NET

- Entity Framework Core
- ASP.NET Core
- WPF
- Windows Forms
- ...

CLR

- CoreCLR
- Base Class Libraries
- JIT & NGEN
- Garbage Collection
- Exception Handling
- Loader von Assemblies

Sechs einfache Fragen die wir uns stellen:

- Wieviel DLLs werden während der Ausführung geladen? Wann?
- Wieviel Methoden hat die Klasse „Person“?
- Wann/Wie wird die Klasse „gejited“ ?
- Wieviel Bytes belegt die Klasse „Person“ ?
- Wieviel Threads gibt es?
- Wer räumt „p“ weg? Wer die Strings? Und Wie? Und Wann?

```
public class Program
{
    private static void Main()
    {
        Person p = new Person()
        {
            Firstname = "Christian",
            Lastname = "Giesswein"
        };
    }
}

internal class Person
{
    public string Firstname { get; set; }
    public string Lastname { get; set; }

    public void SayHello()
    {
        Console.WriteLine("SayHello has been called");
    }
}
```

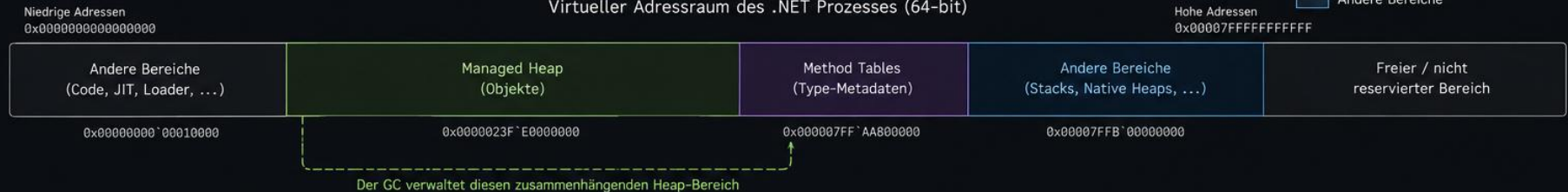
.NET Speicher



GIESSWEIN
Software-Solutions

.NET – Lineare Speicherstruktur

Ein .NET Prozess besitzt einen linearen virtuellen Adressraum



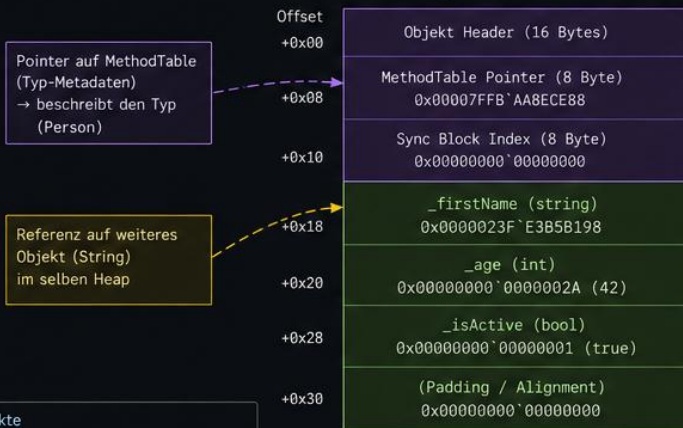
Vergrößerung: Managed Heap (Objekte liegen linear hintereinander)



Detail: Person Objekt @ 0x0000023F`E3B5A2D0

MethodTable (Typ-Metadaten) @ 0x00007FFB`AA8ECE88

EEClass (Typ-Definition) @ 0x00007FFB`AA8ED200



+0x00	Pointer to EEClass
+0x08	Pointer to Parent MethodTable
+0x10	Interface Map
+0x18	vtable Slot 0 (ToString)
+0x20	vtable Slot 1 (Equals)
+0x28	vtable Slot 2 (GetHashCode)
+0x30	...
+0x40	Field Descriptors
+0x48	GC Descriptor
+0x50	Type Flags
+0x58	Component Size
+0x60	...

+0x00	Klassendaten
+0x08	Name ("Person")
+0x10	Namespace ("MyApp.Models")
+0x18	Field Count (3)
+0x20	Method Count (...)
+0x28	...

Beispiel aus WinDbg (SOS)

0:000> !dumpobj /d 0000023FE3B5A2D0

Name: Person
MethodTable: 00007ffbaa8ece88
Size: 56(0x38) bytes

MT	Field	Offset	Type	VT	Attr	Value	Name
....	4000001	18	System.String	0	instance	0000023FE3B5B198	_firstName
....	4000002	20	System.Int32	1	instance	42	_age
....	4000003	24	System.Boolean	1	instance	true	_isActive

Wichtige Punkte

- Der .NET Prozess hat einen linearen virtuellen Adressraum.
- Der GC verwaltet einen zusammenhängenden Heap-Bereich für Objekte.
- Objekte liegen linear im Heap, eines nach dem anderen.
- Jedes Objekt beginnt mit einem 16-Byte Header.
- Der erste Pointer im Header zeigt auf die MethodTable (Typ-Metadaten).
- Felder (Instanzdaten) folgen direkt danach.
- Referenz-Typen (z.B. string) sind weitere Objekte im Heap.

Legende

- Objekt Header / Typ-Metadaten
- Instanzdaten (Felder)
- Andere Objekte (z.B. String)
- Andere Bereiche im Prozess

MethodTable



GIESSWEIN
Software-Solutions

.NET Objekt im Speicher

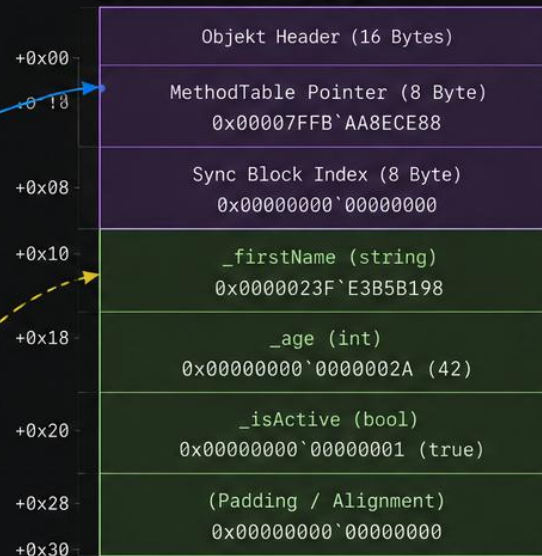
Objekt Referenz (z.B. in einer Variable)

obj (64-bit Prozess)

0x0000023F`E3B5A2D0

Objekt im Heap

Adresse: 0x0000023F`E3B5A2D0



Felder (Instanzdaten)

Beispiel aus WinDbg (SOS)

0:000> !dumpobj /d 0000023FE3B5A2D0

Name: Person

MethodTable: 00007ffbaa8ece88

Size: 56(0x38) bytes

Fields:

MT	Field	Offset	Type	VT	Attr	Value	Name
....	40000001	10	System.String	0	instance	0000023FE3B5B198	_firstName
....	40000002	18	System.Int32	1	instance	42	_age
....	40000003	20	System.Boolean	1	instance	true	_isActive

MethodTable (Typ-Metadaten)

Adresse: 0x00007FFB`AA8ECE88

+0x00	Pointer to EEClass
+0x08	Pointer to Parent MethodTable
+0x10	Interface Map
+0x18	vtable Slot 0 (ToString)
+0x20	vtable Slot 1 (Equals)
+0x28	vtable Slot 2 (GetHashCode)
+0x30	...
+0x40	Field Descriptors
+0x48	GC Descriptor
+0x50	Type Flags
+0x58	Component Size (0 für Referenz-Typen)
+0x60	...

EEClass (Typ-Definition)

Adresse: 0x00007FFB`AA8ED200

+0x00	Klassendaten
+0x08	Name ("Person")
+0x10	Namespace ("MyApp.Models")
+0x18	Field Count (3)
+0x20	Method Count (...)
+0x28	...

Legende

- Objekt Header (Runtime)
- Instanzdaten (Felder)
- Andere Objekte (z.B. String)
- Typ-Metadaten (MethodTable / EEClass)

String Objekt

Adresse: 0x0000023F`E3B5B198

+0x00	MethodTable Pointer 0x00007FFB`AA8EFA10
+0x08	Sync Block Index 0x00000000`00000000
+0x10	Length (int) 0x00000000`00000005 (5)
+0x18	Chars[0] = 'H' 0x00000000`00000048
+0x20	Chars[1] = 'a' 0x00000000`00000061
	...

Wichtige Punkte

- Jedes Objekt beginnt mit einem 16-Byte Header.
- Der MethodTable Pointer zeigt auf Typ-Metadaten.
- Felder folgen direkt nach dem Header (mit Alignment).
- Referenz-Typen (z.B. string) sind eigene Objekte im Heap.

Garbage Collection



GISSWEIN
Software-Solutions

.NET Garbage Collector – Generationsmodell

Der GC verwaltet den verwalteten Heap und sammelt nicht mehr benötigte Objekte automatisch ein.

1) Allokation

Neue Objekte werden im Gen 0 Heap allokiert.

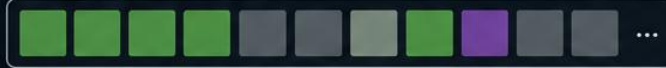
Anwendung
(Dein Code)

Objekt Status

- Live (erreichbar)
- Garbage (nicht erreichbar)
- Pinned

Verwalteter Heap

Gen 0 (junge Generation)



Überlebt Gen 0 → wird zu Gen 1

Gen 1 (mittlere Generation)



Überlebt Gen 1 → wird zu Gen 2

Gen 2 (alte Generation)



Große Objekte (>= 85.000 Bytes) → LOH

LOH (Large Object Heap)



2) Garbage Collection (GC)

Der GC läuft automatisch oder auf Anforderung.

Gen 0 Collection (häufig)

- Sehr schnell
- Nur Gen 0 wird bereinigt
- Überlebende → Gen 1

Gen 1 Collection (gelegentlich)

- Reinigt Gen 0 und Gen 1
- Überlebende → Gen 2

Gen 2 Collection (selten)

- Reinigt alle Generationen
- Teurer als Gen 0/1
- Kompaktiert den Heap

LOH Collection (sehr selten)

- Reinigt nur die LOH
- Keine Kompaktierung
- Nur bei voller Gen 2 GC

3) Roots & Reachability

Der GC findet von den „Roots“ alle erreichbaren Objekte. Nicht erreichbare Objekte werden als Garbage markiert und beim nächsten GC entfernt.

Beispiele für Roots

- Statsicle Felder
- Lokale Variablen (Stack)
- CPU Register
- Handles / Pinned Objects

Ablauf einer Collection (Beispiel: Gen 0)

Vor der Collection



Mark Phase

GC markiert alle erreichbaren Objekte



Sweep Phase

Nicht erreichbare Objekte werden entfernt



Kompaktierung (optional)

Verschiebt verbleibende Objekte



Wichtige Fakten

- Gen 0 Collections sind sehr häufig und schnell.
- Die meisten Objekte sterben jung (in Gen 0).
- Gen 2 Collections sind selten, aber teuer (Stop-The-World).
- LOH speichert große Objekte (>= 85 KB).
- .NET GC ist generational, konzipiert für hohe Performance.



LOH Details

- Objekte >= 85.000 Bytes werden direkt im LOH allokiert.
- LOH wird nicht kompaktiert (kann fragmentieren).
- Wird nur bei einer Gen 2 Collection bereinigt.



Modi des .NET GC

- Workstation GC: Für Client-Anwendungen (Standard für Einzelbenutzer).
- Server GC: Für Server-Anwendungen mit mehr Throughput (mehr Threads).
- Hintergrund GC (optional): Reduziert Pause-Zeiten weiter.



GIESSWEIN
Software-Solutions

Schulungs-, Beratungs- und/oder Projektunterstützung ?!

christian@software.tirol

www.software.tirol

Unterlagen / Beispiele

<https://github.com/softwaretirol/conferences>

Ing. Christian Giesswein, MSc.
Trainings | Consulting | Entwicklung
www.software.tirol